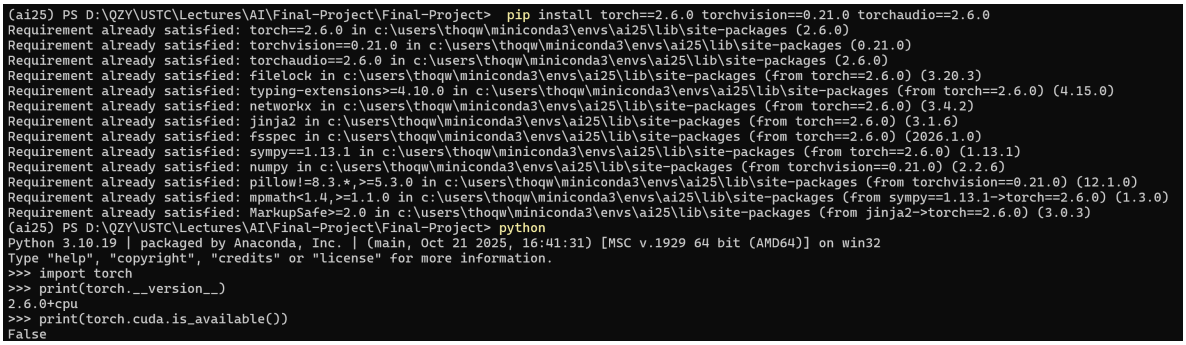
AI final project 报告

邱泽远 PB22000233 刘润🅕 PB24000201 栾登闳 PB24000360

January 31, 2026

1 Torch 配置



```
(ai25) PS D:\QZY\USTC\Lectures\AI\Final-Project\Final-Project>  pip install torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0
Requirement already satisfied: torch==2.6.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (2.6.0)
Requirement already satisfied: torchvision==0.21.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (0.21.0)
Requirement already satisfied: torchaudio==2.6.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (2.6.0)
Requirement already satisfied: filelock in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (4.15.0)
Requirement already satisfied: networkx in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.4.2)
Requirement already satisfied: jinja2 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.1.6)
Requirement already satisfied: fsspec in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (2026.1.0)
Requirement already satisfied: sympy==1.13.1 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (1.13.1)
Requirement already satisfied: numpy in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torchvision==0.21.0) (2.2.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torchvision==0.21.0) (12.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from sympy==1.13.1->torch==2.6.0) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from jinja2->torch==2.6.0) (3.0.3)
(ai25) PS D:\QZY\USTC\Lectures\AI\Final-Project\Final-Project> python
Python 3.10.19 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 16:41:31) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.__version__)
2.6.0+cpu
>>> print(torch.cuda.is_available())
False
```

Figure 1: Torch 环境配置

我们使用 pip 成功安装了 PyTorch，并在 Python 中成功导入 torch，打印版本号和 CUDA 可用性，确认环境配置完成。

2 warmup

在 warmup 部分，我们完成了 MClearning,qlearning 和 sarsa 的算法。以下是对实验结果结果的分析。

2.1 MC Learning

核心思想：Monte Carlo 方法的核心思想是利用**经验平均代替期望**。First-Visit MC 特指在一个回合（Episode）中，对于状态 s（或状态-动作对 (s,a)），只利用该回合中**第一次**访问该状态时产生的累积回报 G_t 来更新价值估计。这保证了样本之间的独立同分布（i.i.d.）特性，从而提供无偏估计。

更新对象：First-Visit MC 主要更新**动作价值函数** $Q(s,a)$。对于每一个状态-动作对 (s,a)，维护两个变量：

- $Returns(s,a)$: 存储所有首次访问该对时的累积回报列表（或累积和）。
- $N(s,a)$: 存储首次访问该对的次数。

更新公式为:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}(G_t - Q(s,a)) \tag{1}$$

这等价于 $Q(s,a) = \text{average}(Returns(s,a))$。

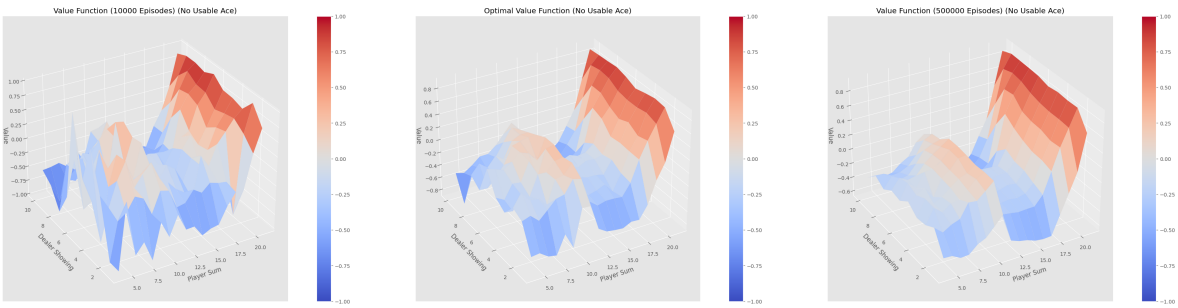
策略改进方式: 使用 ϵ-greedy 策略进行改进:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|} & \text{if } a = \arg\max_{a'} Q(s,a') \\ \frac{\epsilon}{|A(s)|} & \text{otherwise} \end{cases} \tag{2}$$

其中 ϵ 确保了探索性 (Exploration)，使得所有动作都有非零概率被选中，满足蒙特卡洛方法所需的持续探索假设。

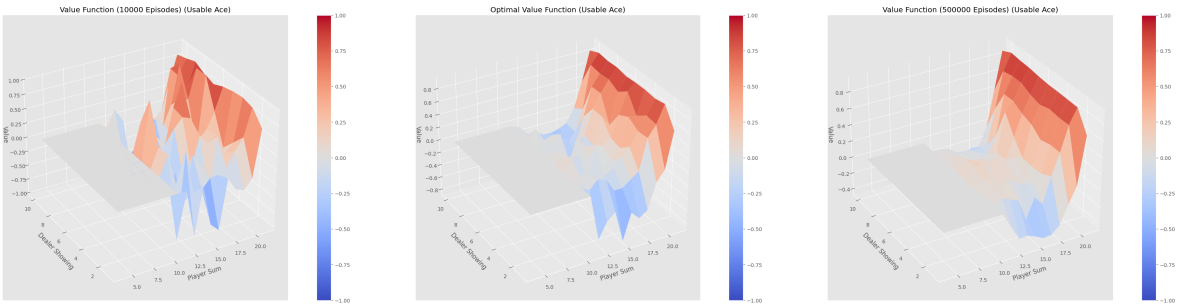
2.1.1 不同训练轮数对比

我们取训练轮数为 10000,100000,500000, 可以得到如下的训练图像:



(a) 轮数 10000 (b) 轮数 100000 (c) 轮数 500000

Figure 2: 不同训练轮数的对比图 (no usable ace)



(a) 轮数 10000 (b) 轮数 100000 (c) 轮数 500000

Figure 3: 不同训练轮数的对比图 (usable ace)

可以看到，在 10000 轮的图像上有很多“尖刺”，并且曲面比较粗糙。这说明这个情况的噪声比较大，并且有可能欠拟合。随着训练轮数的增加，曲面变得逐渐光滑平整。

由于 MC 方法使用采样估计 Q 值，因此，如果样本量不够，估计值将会有很大的方差，并且难以收敛到真实值，导致曲线较为粗糙/不规则。随着样本量的增加，根据大数定律，状态价值的估计值逐渐收敛到真实的期望回报，导致曲线变得较为光滑平整，且接近真实值。

如果有 usable ace，相当于手里的 A 既可以当 1 也可以当 11，这给了玩家第二次机会，而由于 12-16 为 “硬点数”，导致图像在 12-16 区域下降的较为陡峭，因此 usable ace 的价值会略高于 no usable ace 的价值。

2.1.2 first-visit 讨论

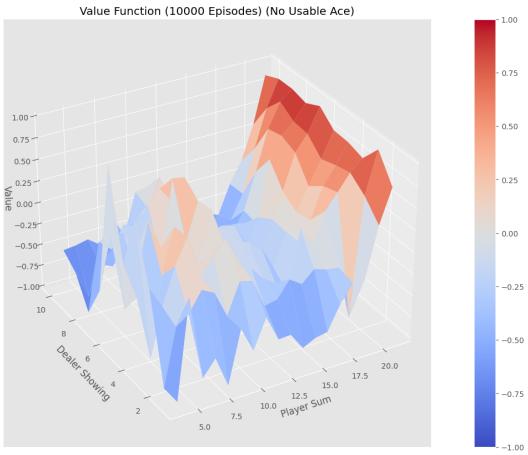
收敛特点

1. 无偏性 (Unbiasedness)：由于每个状态的价值估计仅基于独立的回合样本（首次访问的回报），其期望值严格等于真实价值 $v_\pi(s)$，不存在系统性偏差。
2. 高方差 (High Variance): 估计值依赖于完整的长回合回报，受环境随机性和策略随机性的累积影响，单次采样的波动极大，导致收敛过程震荡明显。
3. 收敛速度: 服从大数定律，标准差随采样次数 N 以 $1/\sqrt{N}$ （中心极限定理保证了标准差为 $\frac{\sigma}{\sqrt{n}}$ 的速度下降。虽然无偏，但由于方差大，通常需要海量样本才能稳定收敛。

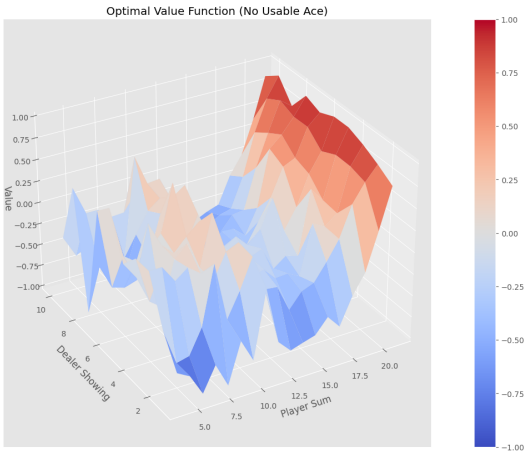
方差来源

1. 回报的随机性：由于 MC 的更新目标为 G_t，这是一个多步的累积量，并且每次动作更新采取 $\epsilon - greedy$ 策略，因此每一步都有随机性且包含多步，因此即使从同一个状态出发，多步累积的随机性也会导致状态的差异特别大。
2. 特定状态的稀疏访问：在 Blackjack 这种状态空间较大的环境中，某些状态（如手中牌是特定组合）可能很难遇到。对于这些 “冷门” 状态，样本量 $N(s)$ 很小，导致平均值的波动（方差）极大。
3. 样本量问题：相对 every visit 而言，first visit 在每一个回合只能获得一个估值，并且有些回合的轮数可能较长，这限制了 first visit 获得足够多的样本，导致方差可能较大。

2.1.3 与 every visit 的对比分析



(a) first visit 10000 轮



(b) every visit 10000 轮

Figure 4: 对比 (no_ace)

理论情况

- **First-Visit**: 由于只取首次访问，它完全忽略了该回合后续再次经过该状态时的回报信息。这使得它对状态转移中的循环（Loops）不敏感，但在样本量少时，估计值**波动变大**。导致稳定性比 every visit 差。
- **Every-Visit**: 利用了更多数据平滑估计值，因此通常表现出更好的稳定性（Variance Reduction）。

实际情况

我们需要注意到 Blackjack 的特殊性：因为牌只会越拿越多，状态只会单向流动（除了 Usable Ace 降级那一下，但也回不到原点数），所以一个 Episode 里几乎不会重复访问同一个 (PlayerSum, DealerCard, UsableAce) 状态。这意味着在 Blackjack 中，first visit 和 every visit 的行为几乎一模一样。这与我们在这两幅图的对比中的情况吻合，事实上，两者之间的差异没有这么大。

2.2 Temporal-Difference Learning

2.2.1 更新差异

SARSA：在更新 $Q(s,a)$ 时，使用的是**实际执行的下一个动作** a'。这意味着它在学习它**当前正在执行的策略**（包括探索步骤），也就是说，sarsa 是 on-policy 的。

更新公式：

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)] \tag{3}$$

特点：

- 考虑到 ϵ-greedy 中的探索风险。
- 在存在危险区域（如悬崖环境）时，SARSA 会倾向于选择更安全的路径，以避免因随机探索掉入陷阱。

Q-learning: Q-learning 在更新 $Q(s,a)$ 时，假设下一步采取的是**最优动作** $\max_{a'} Q(s',a')$，而不管实际执行了什么动作。这意味着它在学习**最优策略**，而行为策略可以是其他的（如 ϵ-greedy）。与 sarsa 不同的是,qlearning 是 off-policy 的。

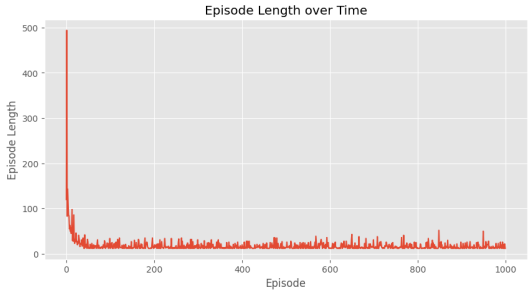
$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{\mathbf{a}'} Q(s',a') - Q(s,a)] \tag{4}$$

特点：

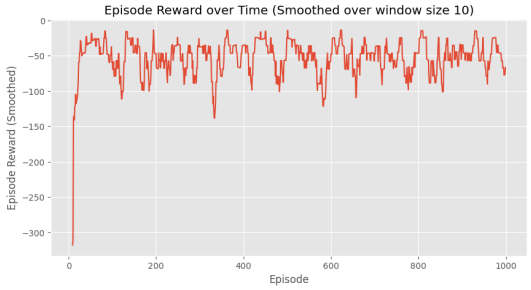
- 直接逼近最优价值函数 Q^*。
- 更加激进（Aggressive），在悬崖环境中会贴着悬崖边走（最优路径），虽然在训练时可能会因探索频繁掉下去，但最终学到的策略是最短路径。

2.2.2 结果分析

取超参数 discount factor = 1.0,alpha = 0.5,epsilon = 0.1，训练轮数等于 1000，我们得到的 qlearning 和 sarsa 的每回合回报/步数的曲线如下图所示：

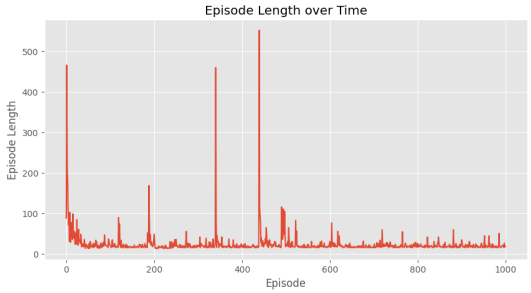


(a) qlearning episode length



(b) qlearning episode reward

Figure 5: qlearning



(a) sarsa episode length

(b) sarsa episode reward

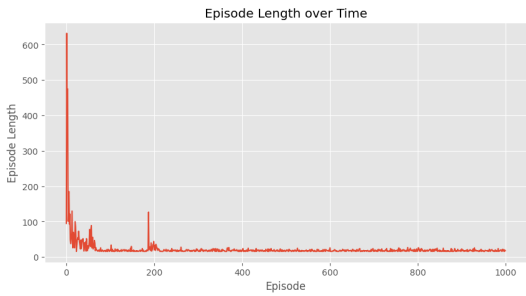
Figure 6: sarsa

分析:

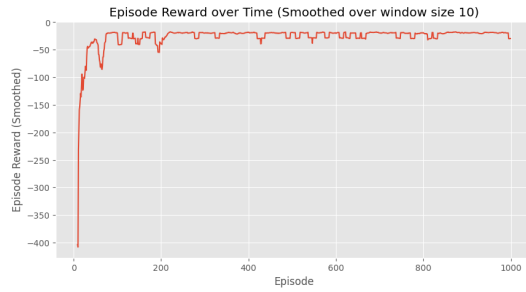
length: 由两幅图可以看到，即使 sarsa 在约 400 左右的点有不稳定的 episode length，两者总体收敛的都比较好，在约 600 过后 episode length 达到了一个较为低的收敛水平。可见，两个算法对于 length 而言都可以较为优秀地收敛。然而，由于 Qlearning 渴望更新的是 off-policy 的 Q，因此其会采取更加激进的策略，在遇到较为“安全且漫长的路径”时，sarsa 不一定渴望像 Qlearning 那样直奔目标，因此该策略在随机性的因素下有概率一直在安全区“打转”，这就解释了为什么 sarsa 比 qlearning 在步数上更有可能出现 length 非常高的点且方差更大。

reward: 分析 reward 的图像可以发现，sarsa 和 qlearning 在 episode 较高时 reward 都在-50 附近，不过 sarsa 收敛得较为平稳，600 轮之后几乎都在-50 之上，然而 qlearning 的波动较大。这是因为 qlearning 采取的是更为激进的策略，因此，其更有可能在路径行走中承担更大的损失/更大的收益，因此其方差更大。

同时，我们也完成了 double qlearning, 效果如下图所示：



(a) double qlearning episode length



(b) double qlearning episode reward

Figure 7: double q learning

可以看到,double qlearning 的效果是所有算法中效果最好的，其最快达到收敛，且收敛于一个较高的值。与 Qlearning 相不同的是，double q learning 的图像收敛之后非常稳定。

我们可以从 Qlearning 对 Q 的估计过高的缺陷来解释这一个问题：由于我们每次估计 Q 值的时候都是有偏差的，有的偏高，有的偏低，而 Qlearning 每次在更新的时候选的都是使得 $Q[next_state][a]$ 最大的 a，因此估计偏高的 Q 更倾向于被选到，这会让我们对 Q 的估计值常常偏高，导致结果不一定准确。同时，如果在 qlearning 的过程中突然选到了 Q 值没有那么高的动作，导致发现先前估计偏高的 Q 值“没有那么好”时，这个 Q 值会迎来暴跌，导致 reward 的不稳定。

因此 double qlearning 的解决方案主要针对 Qlearning 的估值过高问题：这个算法采取两套独立的估计 Q 值的方案 Q1 和 Q2，如果使用 Q1 选择最佳动作，则使用 Q2 估值，如果使用 Q2 选择最佳动作，则使用 Q1 估值，并且在程序中随机选择上面两种情况的一种。由 Q1 和 Q2 的独立性，两者在对同一个动作高估的概率比较小，因此更容易得到准确的估值，同时减轻了图像的波动。

3 主实验

3.1 实验原理

3.1.1 二元零和马尔可夫博弈

二元零和马尔可夫博弈（Two-player Zero-sum Markov Game）指的是：两个玩家在一个随时间变化的环境中进行对抗决策，每一步的选择都会影响下一步局势，且一方获得的收益等于另一方的损失。

- **二元（Two-player）**：只有两个玩家。
- **零和（Zero-sum）**：双方收益相加为 0，

$$r_1(s, a_1, a_2) = -r_2(s, a_1, a_2).$$

- **马尔可夫性（Markov）**：下一状态只依赖当前状态与双方动作，

$$s_{t+1} \sim P(\cdot \mid s_t, a_{1,t}, a_{2,t}).$$

3.1.2 Naive Self-Play

Naive Self-Play 指的是：智能体通过不断与“当前版本的自己”（或最新策略）对战来生成数据并更新策略，从而在对抗环境中逐步变强。

- 设策略为 $\pi_\theta(a \mid s)$，朴素自我博弈的训练过程是反复执行：

$$\text{rollout from } (\pi_\theta \text{ vs. } \pi_\theta) \;\Rightarrow\; \mathcal{D} \;\Rightarrow\; \theta \leftarrow \theta + \Delta\theta.$$

- 优点：实现简单，不需要外部专家数据。
- 常见问题：可能出现循环（策略互相克制）、训练不稳定、遗忘旧对手。

3.1.3 Actor-Critic 方法

Actor-Critic 是一类强化学习方法：用两个模型分工合作来学习策略，**Actor** 负责选动作，**Critic** 负责评估动作好不好。

- **Actor（策略网络）**：$\pi_\theta(a \mid s)$, 用来选择动作.
- **Critic（价值网络）**：$V_\phi(s)$ 或 $Q_\phi(s,a)$, 用来估计回报. 以状态价值为例：

$$V^\pi(s) = \mathbb{E}_\pi\Big[\sum_{t=0}^{\infty} \gamma^t r_t \,\Big|\, s_0 = s\Big].$$

通常用 TD 误差（时序差分误差）近似优势：

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

Actor 通常使用策略梯度更新，利用优势函数（Advantage）降低方差. Actor 的更新方向（最大化期望回报）：

$$\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A_t\big].$$

Critic 的更新通常最小化 TD 误差平方：

$$\mathcal{L}(\phi) = \mathbb{E}\big[\delta_t^2\big].$$

3.2 模型设计

3.2.1 Actor：策略网络（Policy Network）

功能与输入输出

Actor 的目标是生成策略 $\pi(a \mid s)$，使得智能体能够在状态 s 下选择动作 a，并最大化长期累计回报。本实现中：

- 输入：棋盘状态（形状为 (N,N) 或 $(B,1,N,N)$）
- 输出：策略概率分布（形状为 (B,N^2)）

网络结构关键代码

Actor 使用卷积网络提取棋盘空间特征，然后通过全连接层输出 N^2 维的动作打分（logits）。

```python
N = board_size
# feature extractor
self.conv_blocks = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=32,
              kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(in_channels=32, out_channels=64,
              kernel_size=3, padding=1),
    nn.ReLU(),
)

# policy head
self.linear_blocks = nn.Sequential(
    nn.Flatten(),  # shape: 64*N*N
    nn.Linear(64*N*N, 256),
    nn.ReLU(),
    nn.Linear(256, N*N),  # shape: N*N
)
```

Listing 1: Actor 网络结构：卷积特征提取 + 全连接策略头

设计动机

- **使用 CNN：**棋盘是二维结构，卷积层能捕捉局部空间关联（如相邻棋子形态），比纯 MLP 更符合数据结构。
- 3×3 **卷积核 + padding=1：**保持特征图大小不变，使输出仍为 $N \times N$，便于映射到动作空间 N^2。
- **两层卷积逐步增强表达：** 32 通道提取基础局部模式，64 通道组合形成更高层特征。
- **全连接输出 N^2 logits：**每个格子对应一个动作，因此策略输出自然为 N^2 维。

前向传播

Actor 设计中关键的一点是，必须保证非法动作概率为 0，并对合法动作重新归一化。

```python
# 检查 board 的 shape
if len(x.shape) == 2:  # (N,N)
    board = torch.tensor(x).to(device).to(
        torch.float32).unsqueeze(0).unsqueeze(0)
else:  # (B,1,N,N)
    board = torch.tensor(x).to(device).to(torch.float32)

B = board.shape[0]

features = self.conv_blocks(board)
logits = self.linear_blocks(features)  # (B, N*N)

# illegal actions: 不能下在有子区域
legal_mask = (board == 0).view(B, -1)  # (B,N*N)
```

```
masked_logits = logits.masked_fill(~legal_mask, float('-inf'))

output = torch.softmax(masked_logits, dim=1)

no_legal = legal_mask.sum(dim=1) == 0
if no_legal.any():
    output[no_legal] = torch.zeros_like(output[no_legal])

return output
```

Listing 2: Actor 前向传播：处理输入 + 合法动作 mask + 归一化

3.2.2 Critic：价值网络（Q-value Network）

功能与输入输出

Critic 的目标是估计动作价值函数 $Q(s, a)$，用于拟合 Bellman 方程并指导 Actor 更新。

- 输入：棋盘状态 s（形状为 (N, N) 或 $(B, 1, N, N)$）与动作 a（形状为 $(B, 2)$）
- 输出：对应动作的 Q 值（形状为 $(B,)$）

网络结构关键代码

Critic 的特征提取部分与 Actor 类似，但输出表示每个动作对应的 Q 值。

```
N = board_size
# feature extractor
self.conv_blocks = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=32,
              kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(in_channels=32, out_channels=64,
              kernel_size=3, padding=1),
    nn.ReLU(),
)

# value head
self.linear_blocks = nn.Sequential(
    nn.Flatten(),
    nn.Linear(64*N*N, 256),
    nn.ReLU(),
    nn.Linear(256, N*N),
)
```

Listing 3: Critic 网络结构：卷积特征提取 + 全动作 Q 值输出

设计动机

- **输出全动作 Q 值：** 先预测 $Q(s, \cdot) \in \mathbb{R}^{N^2}$，再抽取指定动作对应的 Q 值，计算效率高。
- **与 Actor 相同的 CNN 结构：** 棋盘特征提取方式一致，保证 Critic 也能理解局部空间结构。

- **前向传播**: 索引映射 + gather 抽取指定动作 Q 值

```
indices = torch.tensor(
    [_position_to_index(self.board_size, r, c) for r, c in action]).to(device)

if len(x.shape) == 2:
    board = torch.tensor(x).to(device).to(
        torch.float32).unsqueeze(0).unsqueeze(0)
else:
    board = torch.tensor(x).to(device).to(torch.float32)

features = self.conv_blocks(board)
q_values = self.linear_blocks(features)  # (B, N*N)

output = q_values.gather(1, indices.unsqueeze(1)).squeeze(1)  # (B,)
return output
```

Listing 4: Critic 前向传播：提取 Q(s,a)

设计动机

- **动作坐标到索引的映射：** 对于动作 (r, c)，将其映射到一维索引：

$$k = r \cdot N + c,$$

从而与 N^2 维输出对齐。

- **使用 gather：** gather 能对 batch 内每个样本抽取不同索引的 Q 值，避免 for-loop，提高计算效率。

- **输出维度为 $(B,)$：** 每个样本只对应一个动作的价值估计，符合 $Q(s, a)$ 的定义。

3.3 约束策略分析

在五子棋任务中，Actor 要输出一个策略 $\pi(a \mid s)$,$a \in [N^2]$, 但要符合约束：不能下在有子区域。并且 π 作为一个概率分布，需要满足归一化条件

$$\sum_a \pi(a \mid s) = 1$$

.

由于网络生成的是 logits$\in \mathbb{R}^{\mathbb{N}^{\not\leq}}$，需要通过 softmax 将其映射成概率分布。若 logits$= -\infty$，则对应概率为 0，所以自然地可以通过将所有 board 上有子的位置的 logits 置为负无穷来实现概率为 0。

$$\log \pi(a|s) = \begin{cases} \text{logit}_a & a \in \mathcal{A}_{legal}(s) \\ -\infty & a \notin \mathcal{A}_{legal}(s) \end{cases}$$

其中 $\mathcal{A}_{legal}(s)$ 作为掩码，表示可以落子的位置。对应伪代码：

```
legal_mask = (board == 0).view(B, -1)
masked_logits = logits.masked_fill(~legal_mask, -∞)
```

softmax 函数的性质自动满足概率归一化的性质。但是由于数值计算可能存在误差累积情况，实现过程中对概率分布再次除以所有位置概率的和。

3.4 bug 修复

在本次实验中，我们一共修改了 submission 中 optimizer 的三个部分：

```
#第一处错误：在这里更新时next_qs忘记使用qs.detach(),这里如果没有分离梯度，将会影响到后面critic loss的更新
targets = rewards + gamma * next_qs

#第二处错误：在这里更新optimizer的时候self.actor和self.critic都没有使用step进行更新参数，这样会导致参数永远不会更新，模型永远在原地踏步
self.actor.optimizer.zero_grad()
actor_loss.backward()

self.critic.optimizer.zero_grad()
critic_loss.backward()

#第三处错误（或改进）：actor loss:在源代码中直接使用qs进行loss计算，但是为了降低方差，提高数值稳定性，我们一般可以使用(qs - qs.mean())进行更新。
actor_loss = -torch.mean(torch.log(aimed_policy + eps) * qs.clone().detach())
```

Listing 5: PyTorch 优化器修正前代码

最后得到的代码如下图所示：

```
        targets = rewards + gamma * next_qs.detach()
        critic_loss = nn.MSELoss()(targets, qs)
        indices = torch.tensor(
            [_position_to_index(self.board_size, x, y) for x, y in actions]).to(device)
        aimed_policy = policy[torch.arange(len(indices)), indices]
        actor_loss = - \
            torch.mean(torch.log(aimed_policy + eps) *
                       (qs.detach()-qs.detach().mean()))

        self.critic.optimizer.zero_grad()
        critic_loss.backward()
        self.critic.optimizer.step()


        self.actor.optimizer.zero_grad()
        actor_loss.backward()
        self.actor.optimizer.step()


```

```
        return actor_loss, critic_loss
```

Listing 6: PyTorch 优化器最终代码

3.5 其他问题记录

一开始在训练时训练发生了 crash，前期效果如下图所示：

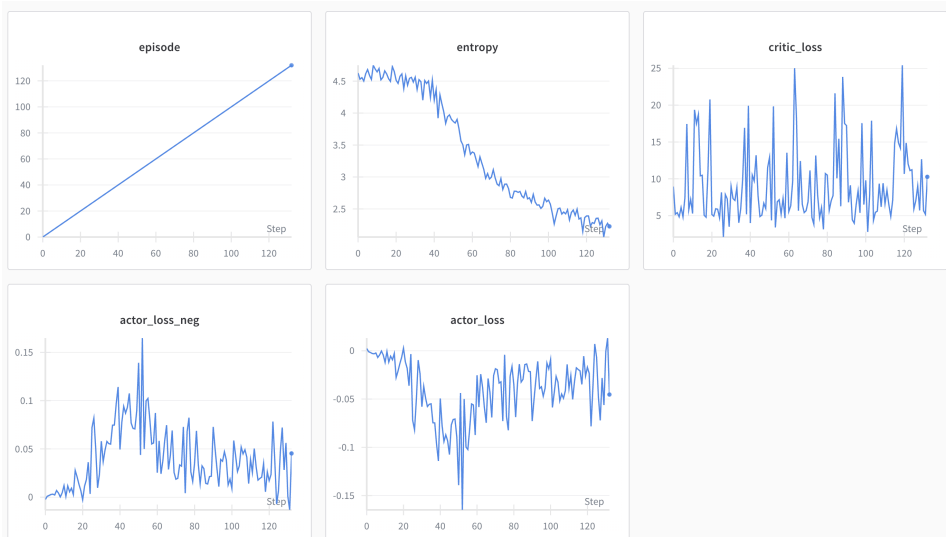


Figure 8: 初始训练效果

旧方案的代码如下：

```
 legal_mask = (board == 0).view(B, -1).float()  # (B,N*N)
masked_logits = logits * legal_mask

exp_probs = torch.exp(masked_logits)
exp_probs = exp_probs * legal_mask
output = exp_probs / (torch.sum(exp_probs, dim=1, keepdim=True) + 1e-8)
```

可以发现，旧代码在非法位置的 logits 置成了 0，这虽然在下一步乘了 legal mask 后对前向传播没有影响，但是会导致梯度回传时对梯度的破坏，并且进行手动归一化的数值稳定性也较差。

因此我们需要修改我们的方案：

```
legal_mask = (board == 0).view(B, -1)  # (B,N*N)
masked_logits = logits.masked_fill(~legal_mask, float('-inf'))

output = torch.softmax(masked_logits, dim=1)

no_legal = legal_mask.sum(dim=1) == 0
if no_legal.any():
    output[no_legal] = torch.zeros_like(output[no_legal])
```

修改后的方案将所有非法的位置都变成负无穷，并且有如下好处：

- **引入 $-\infty$ 屏蔽**: 使用 `masked_fill(~legal_mask, float('-inf'))` 将非法位置的 Logits 设为负无穷。在经过 `softmax` 时，$\exp(-\infty)$ 自然趋近于 0，无需二次掩码。也可以正确的进行前向和后向传播。
- **利用标准 Softmax**: 改用 `torch.softmax(masked_logits, dim=1)`。PyTorch 的 Softmax 实现内部采用了 *Log-Sum-Exp* 技巧，极大地提升了数值稳定性。
- **边界保护机制**: 新增了对无合法动作样本的检测：

```
no_legal = legal_mask.sum(dim=1) == 0
if no_legal.any():
    output[no_legal] = torch.zeros_like(output[no_legal])
```

这确保了在极端情况下，模型的输出行为是确定且安全的 (全零向量)，防止产生 NaN。

3.6 曲线分析

这里我们展示优化后的模型的训练曲线。

3.6.1 Actor Loss 的变化趋势

前 500 个 episode，Actor Loss 快速从接近 0 下降到了-0.5 左右，表面策略开始偏向高优势动作。500-2000 个 episode，均值稳定在-0.5 左右，但方差（波动）逐渐增大，可能是受到 critic 噪声以及探索策略所造成的影响。2000-3000 个 episode 均值进一步降低。但是波动增大，策略更新稳定性下降。

3.6.2 Critic Loss 的变化趋势

主要在 5-15 区间，分布有大量尖峰。有轻微的随训练上升趋势。

可能是由于五子棋状态转移高度非平稳，reward 方差过大，导致 Critic 难以准确估计长期价值。

3.6.3 Policy Entropy 的变化趋势

起始接近 4.6，对应均匀随机策略; 0-1000 episode 快速下降至约 2.0，方差逐渐增大; 1000-3000 episode 缓慢下降至约 1.1，方差逐渐减小。

由于 entropy 反映不确定性，可见策略从“强探索”逐步变为“稳定性”，且后期仍有一定随机性，没有坍缩为确定性策略。

3.7 遇到的疑难问题解答

首先，在观察 actor/critic 的 loss 曲线，可以发现与之前的监督学习不同，其是非常混乱的。经过思考后发现，RL 的 Loss 曲线震荡（Oscillation）是常态，这与监督学习（Supervised Learning）有本质区别。

- 在监督学习中，数据集 D 是固定的。

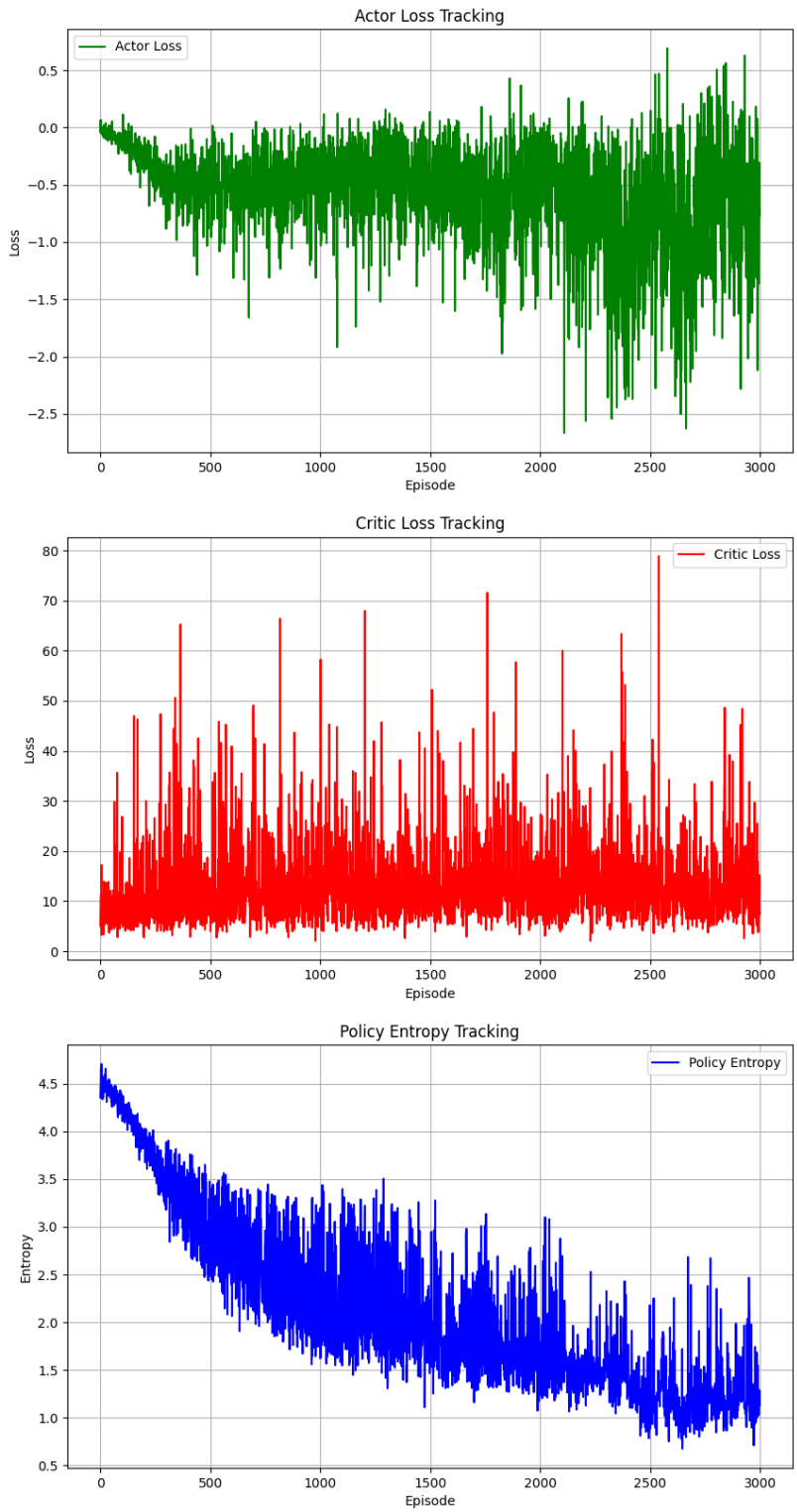


Figure 9: Loss tracker

- 在 RL 中，数据是由当前策略 π_θ 采样生成的。随着网络参数 θ 的更新，策略变了，生成的数据分布也变了。模型一直在追逐一个**移动的目标** (Moving Target)。

Critic Loss 的震荡原因 Critic 的目标是最小化 TD Error: $L = (r + \gamma Q(s', a') - Q(s, a))^2$。

- 当 Agent 发现一个新的高价值策略时，真实回报会突然变大，导致旧的 Q 值估算产生巨大误差（Loss 飙升）。
- 随后 Critic 慢慢学会拟合这个新策略，Loss 下降。
- 紧接着 Agent 又发现了更好的策略，周而复始。因此，Critic Loss 的**周期性脉冲**往往是模型在“顿悟”的体现，而非训练失败。

Actor Loss 的震荡原因 Actor Loss 通常定义为 $-\log \pi(a|s) \cdot A(s, a)$。

- 它的本质是让策略向高优势 ($A > 0$) 的方向移动。
- 随着策略变强，原本的高优势动作变成了“理所当然”的动作，优势函数 A 的数值分布会发生变化。这种动态调整导致 Actor Loss 很难像监督学习那样单调下降。

那么自然会引出下一个问题：entropy 的曲线又可以反映什么？

从 entropy 的表达式可以看出，entropy 反映的是模型做出策略的随机程度，在训练初期，由于是随机初始化的，模型可能觉得下在哪里都差不多，到后期时，训练趋于稳定，模型对自己策略的置信度会变高，这样 entropy 也会变小。因此，entropy 作为系统收敛的指标之一。

同时 entropy 也是易于计算的，经过查阅资料可以发现，我们还可以评估模型的胜率指标：

- **评估方法**: 每隔一定轮次（如 100 轮），让当前模型与一个**固定基准**（如随机策略、贪心策略或早期的自己）对战。
- **收敛信号**: 胜率曲线呈现上升趋势并最终趋于平稳（例如稳定在 90% 以上或 50% 左右的纳什均衡点）。如果胜率还在大幅波动，说明模型策略仍在剧烈调整。

但是该指标面临一个缺点：每次经过若干轮训练后都需要和其他模型进行测试，这样会导致训练耗时很长，时间性价比没有 entropy 高。

3.8 模型优化

在本次实验中，模型优化分为超参数调整和架构创新两个方面：

3.8.1 架构创新

主要架构描述

主干网络用于从原始棋盘状态中提取高层特征。

- **输入**: 12×12 的棋盘状态张量，维度 $(B, 1, 12, 12)$。
- **Stem 层**: 3×3 卷积 (64 通道) $\rightarrow$ BatchNorm $\rightarrow$ LeakyReLU。
- **Residual Tower**: 包含 **5 个**标准残差块 (Residual Block)。

  - 每个 Block 结构: $Conv3 \times 3 \rightarrow BN \rightarrow LeakyReLU \rightarrow Conv3 \times 3 \rightarrow BN \rightarrow$ 残差连接 $\rightarrow LeakyReLU$。

- **输出**: 提取后的特征图，维度 $(B, 64, 12, 12)$。

策略头 (policy head) 负责输出每一步落子的概率分布。引入了 Attention 机制以增强全局感受野。

- **降维卷积**: 1×1 卷积，将通道数从 64 压缩至 **8** (Reduction Dim)。
- **Attention 模块**:
  - Flatten & Transpose → **RMSNorm** (Pre-Norm)。
  - **Multi-Head Attention (MHA)**: 4 Heads, Embedding Dim = 8。
  - Residual Add → **LayerNorm**。
- **全连接模块 (FC Block)**:
  - Flatten → Linear ($8 \times 12 \times 12 \rightarrow 512$) → LayerNorm → LeakyReLU → **Dropout (0.3)**。
  - Output Linear → 维度 144 (对应 12×12 落子位置)。

价值头 (Qhead) 负责评估当前局面的胜率 $Q(s, a)$。结构与策略头类似，但在输出端经过 Tanh 激活。

- 结构与 Policy Head 共享设计模式 (Conv → Attention → FC)。
- **输出**: 经过 **Tanh** 激活的标量值。

optimizer 选择：使用 AdamW 优化器，weight decay = 1e-2

设计动机 由于我们的任务是五子棋棋盘，而五子棋盘是一个二维图形。卷积网络可以提取二维图形中的位置特征以及各个棋子之间的关系等，因此，我们使用一个 residual block 作为一个卷积层组合的块，作为初步的特征提取器。并连续使用五个这样的块，以充分提取特征。

接下来经过一层卷积层压缩特征之后我们使用多头注意力机制提取模型的全局特征。注意，由于前面的卷积层已经充分的提取好了棋盘上的各个位置和各个位置之间的关联等信息，我们不需要像传统的 transformer 那样插入位置编码，而是直接进入 multihead attention 进行前向传播。

同时我们加入残差连接块，这样可以保证训练稳定性，防止梯度爆炸或消失等问题，同时更好地提取特征。

由于训练参数不小，我们需要设计以防止过拟合。通过将 Adam 编辑器修改成 AdamW 编辑器可以有效的防止这一点。同时，我们还在最后的全连接层（事实上最后全连接层的参数量较大，为 $8 \times 12 \times 12 \times 512$ 中引入 dropout。

为了保证训练的稳定，我们在 multihead attention 前加入 RMSnorm，在几乎每个卷积层后加入 BatchNorm，并在全连接层后加入了 layernorm。

设计效果 最终的 reduced param 模型在黑白对局中取得了不俗的效果。

与原始的较为浅层的网络对战，reduced param 的胜率达到了 93%。如下图所示（对局轮数 500 轮）。

```
Black wins: 463, white wins: 35, and ties: 0.
The evaluated winning probability for the black pieces is 0.929718875502008.
100%|
lack wins: 464, white wins: 35, and ties: 0.
The evaluated winning probability for the black pieces is 0.9298597194388778.
Black wins: 465, white wins: 35, and ties: 0.
The evaluated winning probability for the black pieces is 0.93.
100%|
Evaluation finished. Black wins: 465, white wins: 35, and ties: 0.
The evaluated winning probability for the black pieces is 0.93.
```

Figure 10: 与原始模型的对局结果

同时还训练了一个没有添加 attention 层的版本，并将降维卷积层（也就是 attention 层前面的一层）的 reduction dim 提高到了 32.

最终的对局结果为：reduced param 的对局 500 轮全胜，这体现了 attention 层在这个模块中的非常重要的作用。

```
100%| 498/500 [01:06<00:00,  6.19it/s]B
lack wins: 499, white wins: 0, and ties: 0.
The evaluated winning probability for the black pieces is 1.0.
100%| 499/500 [01:06<00:00,  5.69it/s]B
lack wins: 500, white wins: 0, and ties: 0.
The evaluated winning probability for the black pieces is 1.0.
100%| 500/500 [01:06<00:00,  7.49it/s]
Evaluation finished. Black wins: 500, white wins: 0, and ties: 0.
The evaluated winning probability for the black pieces is 1.0.
```

Figure 11: Caption

3.8.2 超参数调整

在认识到这一个架构的优越性之后，我们还尝试增加了这个架构中某些层的参数量：将 reduction dim 从 8 改成 16，并将 dropout 对应的 p 从 0.3 改成 0.1，最终改进的模型参数量约为 3M. 训练 5000 轮。最终结果为：

- 5000 轮的模型对原模型胜率为 0.156
- 取这个模型架构在 4000 轮的实验结果，胜率为 0.448

这告诉我们：模型的参数量的提高不一定代表模型能力的提升，在这个问题以及这个模型架构上，1 2M 的参数量较为合适。与此同时，训练轮数过多也不能带来模型架构的提升，在本问题中，训练了 4000 轮的甚至比训练了 5000 轮的效果更好，训练轮数太多可能导致过拟合。

从训练的图像也可以解释这一点：

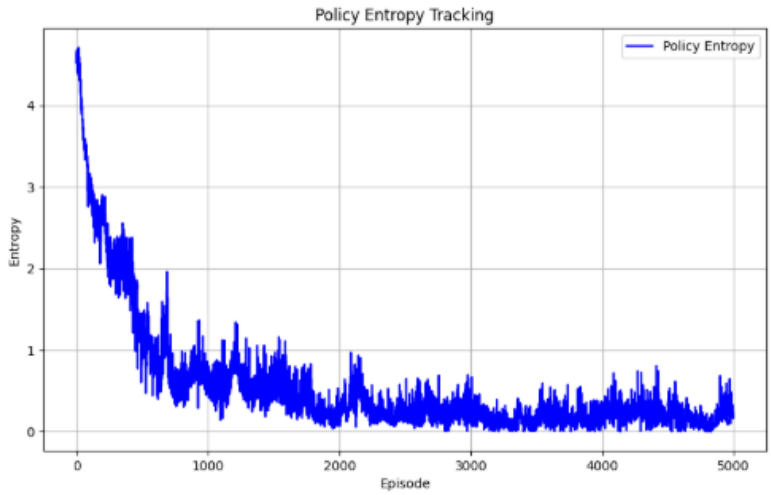


Figure 12: Caption

如图所示，policy entropy 在约为 2000 到 3000 轮后就相对维持在一个较低水平，到 4000 至 5000 轮时的末尾甚至导致曲线的回升，这说明训练轮数过大在这个问题规模上导致的模型的过拟合。

3.9 思考题

若模型权重收敛至 Actor/Critic Loss 最优点，黑白双方策略是否一定达成纳什均衡（Nash Equilibrium）？为什么？

不一定。原因如下：

3.9.1 Actor-Critic 收敛的目标

Actor-Critic 方法的训练目标是最小化 Critic 的 TD 误差（或最大化价值函数估计的准确性），同时优化 Actor 的策略梯度以提升期望回报。即使 Loss 收敛到局部或全局最优点，这只是意味着：

$$\nabla_\theta J(\theta) = 0, \quad \text{且 Critic 误差最小化}$$

并不保证策略满足纳什均衡的定义。

3.9.2 纳什均衡的定义

对于两个玩家的零和博弈，策略 (π^*, σ^*) 是纳什均衡当且仅当：

$$\forall \pi, \quad \mathbb{E}[R(\pi, \sigma^*)] \leq \mathbb{E}[R(\pi^*, \sigma^*)], \quad \forall \sigma, \quad \mathbb{E}[R(\pi^*, \sigma)] \geq \mathbb{E}[R(\pi^*, \sigma^*)]$$

也就是说，每个玩家都无法通过单方面改变策略获得更高回报。Actor-Critic 的最优 Loss 并不能直接保证这一条件成立，尤其是在策略空间较大或存在局部极值时。

3.10 课程反馈

本学期课程内容涵盖了机器学习和人工智能的核心基础，通过本课程系统掌握了监督学习、无监督学习和强化学习等基础概念与方法。

不足与建议：可以增加课堂与作业的衔接，避免作业出现太多超纲内容。